

实验三：跨平台应用开发

一、实验项目基本信息

- 实验编号：d20301035103、d20301009203
- 学时分配：4 学时
- 实验类型：验证型
- 每组人数：6 人
- 对应课程目标：课程目标 2

二、实验目标与成功标准

2.1 实验目标

使用 Flutter 开发业务数据列表页，实现 RESTful API 网络数据请求、JSON 解析与下拉刷新等能力，并在 Android 模拟器上完成运行验证。扩展案例：集成设备传感器（GPS 定位、加速度计）。

2.2 成功标准 (对照)

- ☒ 列表页功能可运行 (Flutter)
- ☒ RESTful API 数据请求正常 (http GET)
- ☒ JSON 数据解析正确 (解析 code/data/items)
- ☒ 下拉刷新功能正常 (Flutter 已实现)

三、需求分析

3.1 业务场景

以“社区公告”为例实现公告列表页，展示公告标题、摘要、发布时间、发布方；支持下拉刷新与滚动加载更多，用于模拟社区/校园新闻通知类业务模块。

3.2 数据模型

课程示例给出返回结构如下：

```
{
  "code": 200,
  "data": {
    "items": [
      {
        "id": 1,
        "title": "智慧校园更新通知",
        "content": "系统升级通知...",
        "author": "管理员",
        "time": "2024-01-01 10:00:00"
      }
    ]
  }
}
```

```
}  
}
```

本次 Flutter 实现使用“社区公告”字段命名，当前为本地模拟数据，后续接入接口时建议在 `fromJson` 中映射字段 (如 `author -> publisher`, `time -> publishedAt`) 。

四、开发环境与工具

4.1 软件环境

- 操作系统：Windows
- Flutter：3.43.0 (master)
- Android Studio：用于安装 SDK/NDK、运行模拟器
- Android SDK 路径：`D:\APP\Android\sdk`

4.2 项目路径

- Flutter 项目目录：`D:\APP\Android\pros\flutter_notice_list`

五、Flutter 模块实现过程

5.1 创建 Flutter 项目

```
flutter create flutter_notice_list  
cd flutter_notice_list
```

5.2 公告列表页实现（界面 + 下拉刷新 + 加载更多）

实现文件：

- `lib/main.dart`

主要实现点：

1. 首页为公告列表页 `AnnouncementListPage`
2. 列表使用 `RefreshIndicator` 实现下拉刷新
3. 监听滚动位置，接近底部触发 `_loadMore()` 实现加载更多
4. 置顶公告展示“置顶”标签；列表项展示标题、摘要、日期与发布方
5. 空态/错误态：首屏加载失败显示重试按钮；加载更多失败显示提示

5.3 数据来源说明

当前实现优先请求 RESTful API 获取公告列表；当接口不可用时，为保证页面可运行与演示效果，会回退显示示例数据，并在列表顶部提示“接口不可用，当前显示示例数据”。

5.4 测试验证（Flutter）

执行静态分析与单测：

```
flutter analyze  
flutter test
```

测试文件：

- `test/widget_test.dart`：验证应用能启动、先显示加载态、随后加载出公告列表并包含置顶项。

六、运行与调试记录（Android 模拟器）

6.1 模拟器运行

在项目目录运行：

```
flutter run
```

6.2 常见问题：NDK 安装失败

现象：

- Gradle 构建时提示安装 NDK (Side by side) `28.2.13676358` 失败，或提示 NDK 目录缺少 `source.properties`

原因：

- NDK 下载/解压不完整，导致本地目录缺文件，Android Gradle Plugin 配置阶段失败。

处理方式：

1. 删除损坏目录：`D:\APP\Android\sdk\ndk\28.2.13676358`
2. Android Studio -> Settings -> Languages & Frameworks -> Android SDK -> SDK Tools
3. 勾选 NDK (Side by side) 并启用 Show Package Details，安装 `28.2.13676358`
4. 重新执行 `flutter run`

七、与实验要求对齐的改进计划（RESTful + JSON 解析）

为满足“RESTful API 网络请求 + JSON 解析”要求，本次已完成以下实现：

1. 在 `pubspec.yaml` 添加依赖：
 - `http`
 - `provider`
2. 使用 `http.get(Uri.parse(url))` 请求公告接口（分页参数：`page`、`pageSize`）
3. `jsonDecode(...)` 解析响应，支持从 `data.items` 或 `items` 提取列表
4. 通过 `Announcement.fromJson(Map<String, dynamic>)` 完成字段映射与容错解析：
 - `author/publisher` 映射为发布方
 - `time/publishedAt` 映射为发布时间
 - `content/summary` 映射为摘要
 - `pinned/isPinned` 映射为置顶标记

5. 刷新与分页:
- 下拉刷新: 请求第一页并替换列表
 - 加载更多: 请求下一页并追加到列表
6. 异常容错:
- 网络异常 / 非 200: 提示错误或回退示例数据 (用于演示)
 - 空数据: 展示空列表/无更多提示

7.1 接口地址配置

默认接口地址为课程示例的占位地址，实际运行时建议通过启动参数注入真实接口：

```
flutter run --dart-define=NEWS_API_URL=http://10.0.2.2:3000/news
```

说明：

- Android 模拟器访问电脑本机服务可使用 10.0.2.2
- 接口需返回 JSON，且能解析出 data.items（与课程示例结构一致）

八、系统设计图（框架图 / 类图 / 顺序图）

8.1 框架图（分层架构）

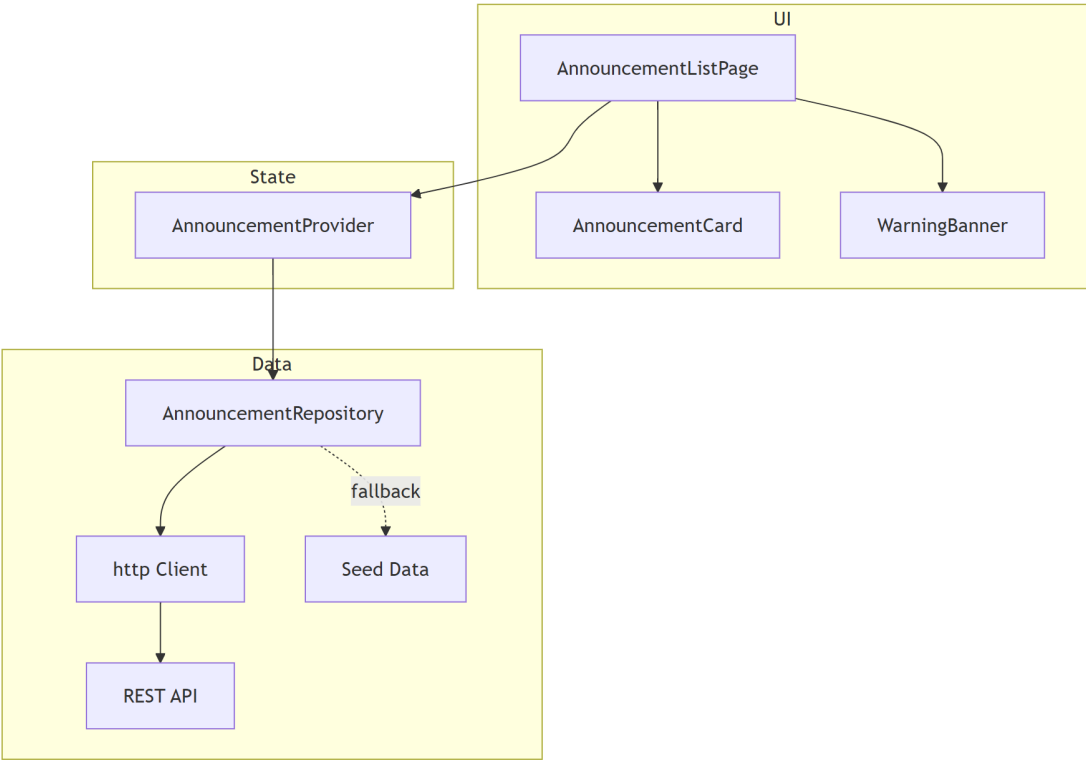


图 1-1 Flutter 社区公告模块分层架构（框架图）

- UI 层负责展示与交互：列表页、列表项卡片、接口回退提示条。
- 状态层使用 Provider（ChangeNotifier）集中管理：loading、error、分页与数据列表。

- 数据层负责网络请求与解析：通过 `http.Client` 请求 RESTful API，失败时可回退示例数据，保证演示可运行。

8.2 类图（核心类关系）

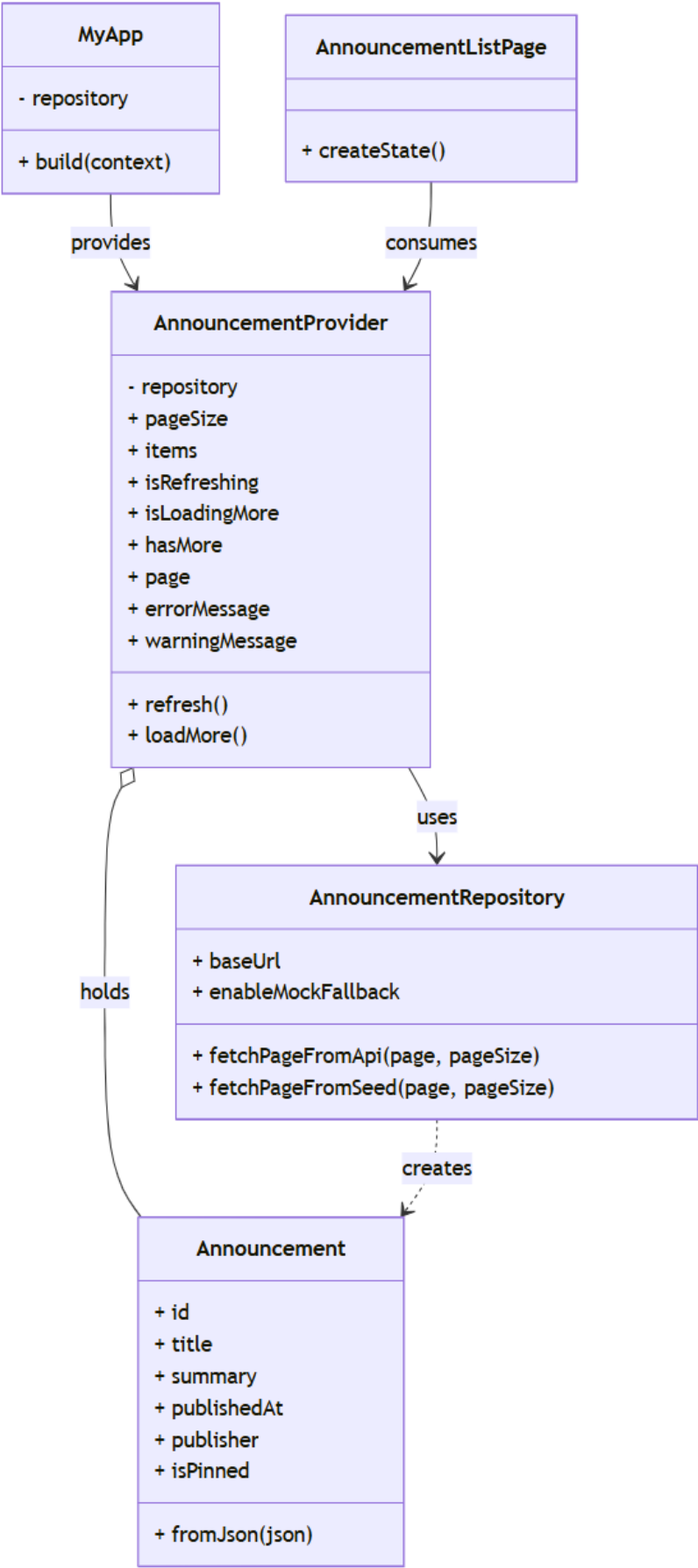


图 1-2 Flutter 社区公告模块类图

- MyApp 负责注入 AnnouncementProvider，并启动 MaterialApp。
- AnnouncementListPage 通过 Consumer 监听 Provider 变化，刷新 UI。
- AnnouncementProvider 聚合业务状态并调用 AnnouncementRepository 获取数据。
- AnnouncementRepository 负责请求 API、解析 JSON 并构造 Announcement 实体。

8.3 顺序图（首屏加载 + 回退逻辑）

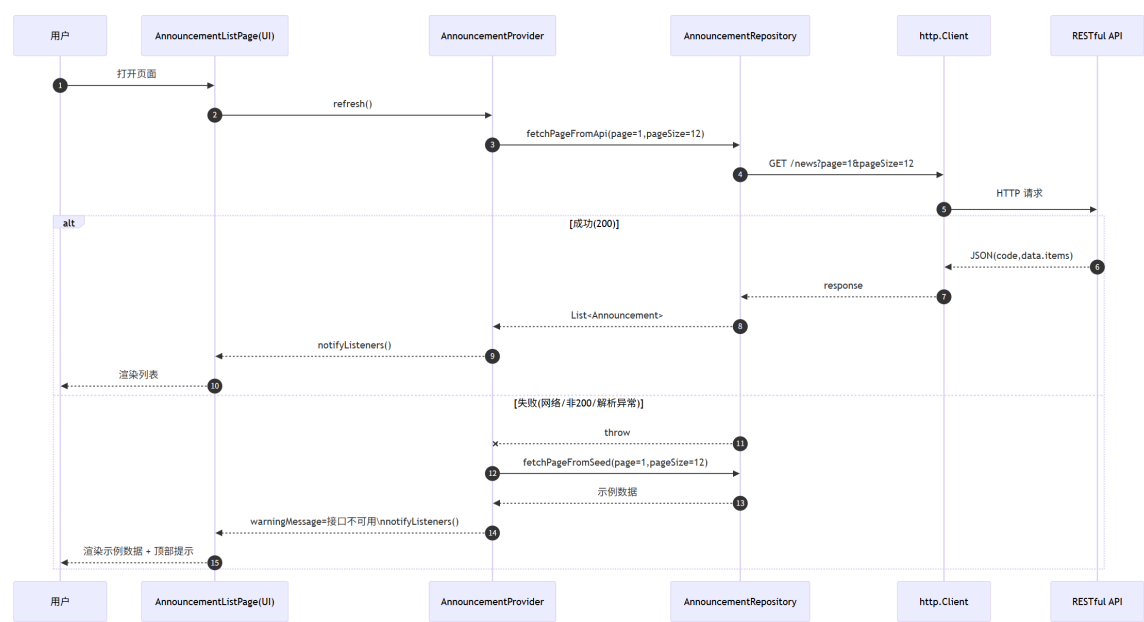


图 1-3 首屏加载与接口回退顺序图

- 页面打开后触发 refresh()：先清理状态并进入 loading。
- Repository 发起 GET 请求并解析 JSON 成 Announcement 列表。
- 成功：Provider notifyListeners()，页面渲染列表。
- 失败：Provider 触发回退（示例数据）并显示顶部提示，保证可演示。

九、个人体会（Flutter）

1. Flutter 通过组件组合快速搭建页面结构， Scaffold + AppBar + ListView 的搭配能快速完成列表型业务页面。
2. RefreshIndicator 能直接覆盖常见的下拉刷新需求，配合异步请求即可实现一致的交互体验。
3. 分页加载在 Flutter 中可以通过监听 ScrollController 的滚动位置实现，逻辑清晰，但需要注意避免重复触发与页面销毁后的状态更新。
4. Android 构建依赖 SDK/NDK 环境较多，首次运行可能出现组件缺失或下载不完整的问题，需要结合 Android Studio 的 SDK Manager 进行修复。

十、实验总结

1. 移动端列表功能的核心由“数据获取 + JSON 解析 + 列表渲染 + 刷新/分页交互”组成。
2. 本次使用 Flutter 完成了公告列表页的 UI 展示、下拉刷新与加载更多交互，并通过测试用例验证基础加载流程。
3. 实验要求中的“RESTful API + JSON 解析”是关键验收点，当前已完成 UI 与交互验证，后续接入真实接口并实现 fromJson 解析即可满足完整标准。

十一、实验截图



图 2-1 社区公告列表页
(首屏)

图 2-2 下拉刷新效果

图 2-3 滚动加载更多与
“没有更多了”提示

图 2-4 接口不可用时回
退示例数据提示